XIAMEN UNIVERSITY MALAYSIA

Course Code	:	SWE 402
Course Name	:	Data Mining
Lecturer	:	Abdullah Talal Ali Ahmed
Academic Session	:	2025/04
Assessment Title	:	Assignment
Submission Due Date	:	20/6/2025

Prepared by :

Student ID	Student Name
DSC2304093	Bong Xuan Jing

Date Received : 12/6/2025

Feedback from Lecturer:

Mark:

XIAMEN UNIVERSITY MALAYSIA

Own Work Declaration

I/We acknowledge that my/our work will be examined for plagiarism or any other form of academic misconduct, and that the digital copy of the work may be retained for future comparisons.

I/We confirm that all sources cited in this work have been correctly acknowledged and that I/we fully understand the serious consequences of any intentional or unintentional academic misconduct.

In addition, I/we affirm that this submission does not contain any materials generated by AI tools, including direct copying and pasting of text or paraphrasing. This work is my/our original creation, and it has not been based on any work of other students (past or present), nor has it been previously submitted to any other course or institution.

Signature: 

Date: 12/6/2025

Section A: Basic Python

1. Write a Python script to solve each of the following problems:

   a. Given the variables:

   Country = "Malaysia"

   Population= 33000000

   Use the print function to display the following string based on the format below.

   The population of Malaysia is 33,000,000.

   [1 Marks]

```
#A
country = "Malaysia"
population = 33000000
print(f"The population of {country} is {population:,}.")
```

```
The population of Malaysia is 33,000,000.
```

I declare a variable named country and assign a value “Malaysia” to it. Then, I declare a variable named population and assign a value “33000000” to it. Using the print function to print them out with :,(Separate thousands of integers with commas).

   b. Given a list of integers:

   numbers = [12, 15, 18, 21, 24, 27, 30, 33, 36, 39]

   - Write a Python Script to print the min, max, sum of the list.
   - Write a Python Script to print [15, 18, 21] from the list.
   - Write a Python script to calculate the sum of all even numbers in the list.

   [2 Marks]

```python
#B
numbers = [12, 15, 18, 21, 24, 27, 30, 33, 36, 39]
print(f"Minimum of the list is {min(numbers)}")
print(f"Maximum of the list is {max(numbers)}")
print(f"Sum of the list is {sum(numbers)}")

print(numbers[1:4])

sum_of_even = 0
for x in numbers:
    if x % 2 == 0:
        sum_of_even += x

print(f"Sum of all even numbers is {sum_of_even}")
```

```
Minimum of the list is 12
Maximum of the list is 39
Sum of the list is 255
[15, 18, 21]
Sum of all even numbers is 120
```

Using min(), max(), sum() function to find minimum, maximum, summation of list.

[1, 4] indicates indices from 1 to 3. (x % 2 == 0) can find the even numbers, and sum them up.

c. Given the following nested dictionary:

```python
data = {
    'key1': [1, 2, 3,
         {'nested_key': ['a', 'b', 'c',
                         {'target': [10, 20, 30, 'hello']}]}]
}
```

- Use indexing to grab nested_key nested dictionary.
- Use indexing to grab the word “hello.”
- Change the word “hello” to 40.

```
#C
data = {
    'key1': [1, 2, 3,
        {'nested_key': ['a', 'b', 'c',
                {'target': [10, 20, 30, 'hello']
                }
            ]
        }
    ]
}

#Use indexing to grab nested_key nested dictionary.
nested_dict = data['key1'][3]
print(nested_dict)

#Use indexing to grab the word "hello."
hello = data['key1'][3]['nested_key'][3]['target'][3]
print(hello)

#Change the word “hello” to 40.
hello = 40
print(data)
```

```
{'nested_key': ['a', 'b', 'c', {'target': [10, 20, 30, 'hello']}]}
hello
{'key1': [1, 2, 3, {'nested_key': ['a', 'b', 'c', {'target': [10, 20, 30, 'hello']}]}]}
```

nested_dict = data['key1'][3] can grab nested_key nested dictionary. hello = data['key1'][3]['nested_key'][3]['target'][3] can grab the word “hello” and store it to a variable hello. Finally change the value stored in hello to 40.

d. Write a Python function that takes a string as input and returns the number of vowels (a, e, i, o, u) in the string. Test your function with the following strings:

- "Artificial Intelligence"
- "Data Management"

[2 Marks]

```
#D
def count_vowels(s):
    count = 0
    for x in s:
        if x=='a' or x=='e' or x=='i' or x=='o' or x=='u' or x=='A' or x=='E' or x=='I' or x=='O' or x=='U' :
            count += 1
    print(f"The number of vowels in the string is: {count}")

string = input("Enter your string: ")
count_vowels(string)
string = input("Enter your string: ")
count_vowels(string)
```

```
Enter your string:  Artificial Intelligence
The number of vowels in the string is: 10
Enter your string:  Data Management
The number of vowels in the string is: 6
```

Using a for loop to traverse each alphabet in string, if it is vowel, add 1 to the variable count.

e. Using the NumPy module, solve the following tasks:

- Create an array of 10 zeros.
- Create an array of all odd numbers between 10 and 50.
- Create a 3x3 identity matrix.

```
#E
import numpy as np
array1 = np.zeros(10)
print(array1)
array2 = np.arange(11, 51, 2) #(start, end (not included), step)
print(array2)
array3 = np.array([[1,0,0],[0,1,0],[0,0,1]])
print(array3)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
[11 13 15 17 19 21 23 25 27 29 31 33 35 37 39 41 43 45 47 49]
[[1 0 0]
 [0 1 0]
 [0 0 1]]
```

Create an array of 10 zeros by using np.zeros(). Create an array of all odd numbers between 10 and 50 by using array2 = np.arange(11, 51, 2). Create a 3x3 identity matrix by using np.array().

Section B: Data Analysis

1. You are given a dataset named **student_data.csv**. This dataset contains information about students, including their names, ages, grades, and subjects. Write a Python script to perform the following tasks:
    a. Import Pandas into your Python program and store the student data in a Pandas DataFrame. Display the first five rows of records.

[1 Marks]

```
#A
import pandas as pd
file_loaction = r"C:\University\Year 3 Sem 1\Data Mining\Assignment\student_data.csv"
df = pd.read_csv(file_loaction)
display(df.head(5))
```

	Name	Age	Grade	Subject
0	Zhang Lei	21.0	2.62	Biology
1	Nadia Binti Hassan	20.0	3.14	Art
2	Khalid Nasir	16.0	3.27	Physics
3	Chen Jie	16.0	3.12	Physics
4	Mohd Faiz	NaN	2.24	Biology

Pd.read_csv() can read csv file and store in df. Using df.head(5) to display first 5 rows.

b. Find and display the number of missing values in each column. Use different ways to handle missing values.

[3 Marks]

```
print(df.isnull().sum())
```

```
Name       0
Age        1
Grade      1
Subject    1
dtype: int64
```

```
df.dropna(subset = ['Subject'], axis=0, inplace=True)
df.reset_index(drop=True, inplace=True)

avg_age = df["Age"].astype("float").mean(axis=0)
df["Age"] = df["Age"].replace(np.nan, avg_age)

avg_grade = df["Grade"].astype("float").mean(axis=0)
df["Grade"] = df["Grade"].replace(np.nan, avg_grade)


print(df.isnull().sum())
```

```
Name       0
Age        0
Grade      0
Subject    0
dtype: int64
```

Drop the whole row with missing value in column Subject. Replace the missing value by mean in column Age and Grade.

c. Discover and show the following information:

i. The average age of students.

```
#C
#i. The average age of students.
print(f"The average age of students is {df['Age'].mean()}.")
```

```
The average age of students is 18.816326530612244.
```

ii. The name of the student with the highest grade.

```
#ii. The name of the student with the highest grade.
highest_grade_row = df.loc[df['Grade'].idxmax()]
print(f"The name of the student with the highest grade is {highest_grade_row['Name']}.")
```

```
The name of the student with the highest grade is Azman Hakim.
```

iii. The number of students who are older than 20.

```
#iii. The number of students who are older than 20.
older_than_20 = df[df['Age'] > 20]
count = len(older_than_20)
print(f"The number of students who are older than 20 is {count}.")
```

```
The number of students who are older than 20 is 33.
```

iv. The percentage of students who have a grade above 3.8.

```
#iv. The percentage of students who have a grade above 3.8.
grade_A = df[df['Grade'] > 3.80]
percentage = (len(grade_A) / len(df)) * 100
print(f"The percentage of students who have a grade above 3.8 is {percentage:.2f} %.")
```

```
The percentage of students who have a grade above 3.8 is 7.07 %.
```

v. The most common subject among students.

```
#v. The most common subject among students.
most_common_subject = df['Subject'].mode()[0]
print(f"The most common subject among students is {most_common_subject}.")
```

```
The most common subject among students is Computer Science.
```

vi. Who is the youngest student?

```
#vi. Who is the youngest student?
youngest_student_row = df.loc[df['Age'].idxmin()]
print(f"The youngest student is {youngest_student_row['Name']}.")
```

```
The youngest student is Ahmad Hafiz.
```

vii. What is the average grade per subject?

```
#vii. What is the average grade per subject?
average_grade_per_subject = df.groupby('Subject')['Grade'].mean().round(2)
print(f"The average grade per subject is: ")
display(average_grade_per_subject)
```

```
The average grade per subject is:
Subject
Art                 3.01
Biology             3.02
Chemistry           3.37
Computer Science    2.89
Economics           3.14
English             3.15
Geography           2.85
History             3.20
Mathematics         2.90
Physics             3.00
Name: Grade, dtype: float64
```

viii. How many students are enrolled in each subject?

```
#viii. How many students are enrolled in each subject?
students_per_subject = df['Subject'].value_counts()
print("Number of students enrolled in each subject: ")
display(students_per_subject)
```

```
Number of students enrolled in each subject:
Subject
Physics             14
Computer Science    14
Geography           12
Biology             11
Art                 11
Mathematics         11
History              9
Chemistry            7
English              5
Economics            5
Name: count, dtype: int64
```

ix. Who are the students with grades below 2.5?

```
#ix. Who are the students with grades below 2.5?
grade_C = df[df['Grade'] < 2.50]['Name']
print("Students with grades below 2.5: ")
print(grade_C.to_string(index=False))
```

```
Students with grades below 2.5:
         Mohd Faiz
            Li Wei
          Chen Jie
Nadia Binti Hassan
        Zainab Ali
          Zhou Lin
       Roslan Amir
          Zhao Rui
      Hana Youssef
          Zhou Lin
        Zainab Ali
       Aisha Ahmed
       Omar Khaled
            Xu Yan
       Azman Hakim
      Samira Fadel
            Li Wei
         Mohd Faiz
       Aisha Ahmed
            Li Wei
       Aisha Ahmed
            Wu Hao
         Mohd Faiz
        Zainab Ali
      Yusuf Kareem
```

x. What is the median grade of students?

```
#x. What is the median grade of students?
median_grade = df['Grade'].median()
print(f"The median grade of students is {median_grade}")
```

```
The median grade of students is 3.14
```

Section C: Data Preprocessing

Using the **Titanic** dataset from Seaborn, answer the following questions.

1. Data Preprocessing and Exploration:

   f. Import the necessary Python libraries and load the Titanic dataset from Seaborn.

   g. Display the first five rows of the dataset and the last ten rows.

```
#1 #f,g
import seaborn as sns
import matplotlib.pyplot as plt
import numpy as np
df2 = sns.load_dataset('titanic')
display(df2.head(5))
display(df2.tail(10))
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
0	0	3	male	22.0	1	0	7.2500	S	Third	man	True	NaN	Southampton	no	False
1	1	1	female	38.0	1	0	71.2833	C	First	woman	False	C	Cherbourg	yes	False
2	1	3	female	26.0	0	0	7.9250	S	Third	woman	False	NaN	Southampton	yes	True
3	1	1	female	35.0	1	0	53.1000	S	First	woman	False	C	Southampton	yes	False
4	0	3	male	35.0	0	0	8.0500	S	Third	man	True	NaN	Southampton	no	True

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class	who	adult_male	deck	embark_town	alive	alone
881	0	3	male	33.0	0	0	7.8958	S	Third	man	True	NaN	Southampton	no	True
882	0	3	female	22.0	0	0	10.5167	S	Third	woman	False	NaN	Southampton	no	True
883	0	2	male	28.0	0	0	10.5000	S	Second	man	True	NaN	Southampton	no	True
884	0	3	male	25.0	0	0	7.0500	S	Third	man	True	NaN	Southampton	no	True
885	0	3	female	39.0	0	5	29.1250	Q	Third	woman	False	NaN	Queenstown	no	False
886	0	2	male	27.0	0	0	13.0000	S	Second	man	True	NaN	Southampton	no	True
887	1	1	female	19.0	0	0	30.0000	S	First	woman	False	B	Southampton	yes	True
888	0	3	female	NaN	1	2	23.4500	S	Third	woman	False	NaN	Southampton	no	False
889	1	1	male	26.0	0	0	30.0000	C	First	man	True	C	Cherbourg	yes	True
890	0	3	male	32.0	0	0	7.7500	Q	Third	man	True	NaN	Queenstown	no	True

Using sns.load_dataset to load Titanic dataset from Seaborn. Using df.head(5) and df.tail(10) to display the first five rows of the dataset and the last ten rows.

h. Display a summary of the dataset, including column types and missing values.

```
#h
df2.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
```

By using .info() function, we can get a clear understanding of the basic information of the dataset, like number of columns, column names, non-null count (not a missing value), and data types.

```
df2.describe()
```

	survived	pclass	age	sibsp	parch	fare
count	891.000000	891.000000	714.000000	891.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594	32.204208
std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693429
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000000
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910400
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454200
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000000
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329200

By using .describe() function, we can get the summary statistics for numerical columns.

```
#h
df2.dtypes
```

```
survived          int64
pclass            int64
sex              object
age             float64
sibsp             int64
parch             int64
fare            float64
embarked         object
class          category
who              object
adult_male         bool
deck           category
embark_town      object
alive            object
alone              bool
dtype: object
```

```
print(df2.isnull().sum())
```

```
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

Now, we have found that there are 177, 2, 688, and 2 missing values in the column age, embarked, deck, and embark_town respectively. We try to handle them with different way later.

i. Handle missing values using at least three different ways.

```
#i
#replace by mean
avg_age = df2["age"].mean()
df2['age'] = df2['age'].replace(np.nan, avg_age)
```

```
df2['embarked'].value_counts()
```

```
embarked
S    644
C    168
Q     77
Name: count, dtype: int64
```

```
#replace by the most frequent
df2['embarked'] = df2['embarked'].replace(np.nan, 'S')
df2['embarked'].value_counts()
```

```
embarked
S    646
C    168
Q     77
Name: count, dtype: int64
```

```
df2['deck'].value_counts()
```

```
deck
C    59
B    47
D    33
E    32
A    15
F    13
G     4
Name: count, dtype: int64
```

To handle column age, I replace the missing values by mean of the age.

To handle column embarked, I replace the missing value by the most frequent (S).

To handle column deck, since there are 688 missing values while the size of the dataset is 891 rows, I decide to drop the whole column.

To handle column embark_town, I replace the missing value by the most frequent (Southampton).

```
#drop the whole column
df2.drop(columns=['deck'], inplace=True)
```

```
df2['embark_town'].value_counts()
```

```
embark_town
Southampton    644
Cherbourg      168
Queenstown      77
Name: count, dtype: int64
```

```
#replace by the most frequent
df2['embark_town'] = df2['embark_town'].replace(np.nan, 'Southampton')
df2['embark_town'].value_counts()
```

```
embark_town
Southampton    646
Cherbourg      168
Queenstown      77
Name: count, dtype: int64
```

```
print(df2.isnull().sum())
```

```
survived       0
pclass         0
sex            0
age            0
sibsp          0
parch          0
fare           0
embarked       0
class          0
who            0
adult_male     0
embark_town    0
alive          0
alone          0
dtype: int64
```

Now the dataset has no missing value anymore.

j. Convert categorical variables into numerical representations where applicable.

```
#drop redundant column
redundant_columns = ['embark_town', 'alive', 'who', 'class', 'adult_male']
df2.drop(columns = redundant_columns, axis = 1, inplace = True)
df2
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	alone
0	0	3	male	22.000000	1	0	7.2500	S	False
1	1	1	female	38.000000	1	0	71.2833	C	False
2	1	3	female	26.000000	0	0	7.9250	S	True
3	1	1	female	35.000000	1	0	53.1000	S	False
4	0	3	male	35.000000	0	0	8.0500	S	True
...	...	...	...	...	...	...	...	...	...
886	0	2	male	27.000000	0	0	13.0000	S	True
887	1	1	female	19.000000	0	0	30.0000	S	True
888	0	3	female	29.699118	1	2	23.4500	S	False
889	1	1	male	26.000000	0	0	30.0000	C	True
890	0	3	male	32.000000	0	0	7.7500	Q	True

891 rows × 9 columns

I drop the redundant columns first.

```
# Convert 'alone' from boolean to int (True->1, False->0)
df2['alone'] = df2['alone'].astype(int)

numeric_columns = df2.select_dtypes(include=['float64', 'int64']).columns
numeric_columns
```

```
Index(['survived', 'pclass', 'age', 'sibsp', 'parch', 'fare'], dtype='object')
```

```
import pandas as pd
#j. Convert categorical variables into numerical representations where applicable.
# One-Hot Encoding
binary_categorical_columns = ['sex']  # Columns with two categories
multi_categorical_columns = ['embarked']  # Columns with more than two categories

df2_multi_encoded = pd.get_dummies(df2[multi_categorical_columns], dtype = int)
df2_binary_encoded = pd.get_dummies(df2[binary_categorical_columns], dtype = int,
                                    drop_first = True)
```

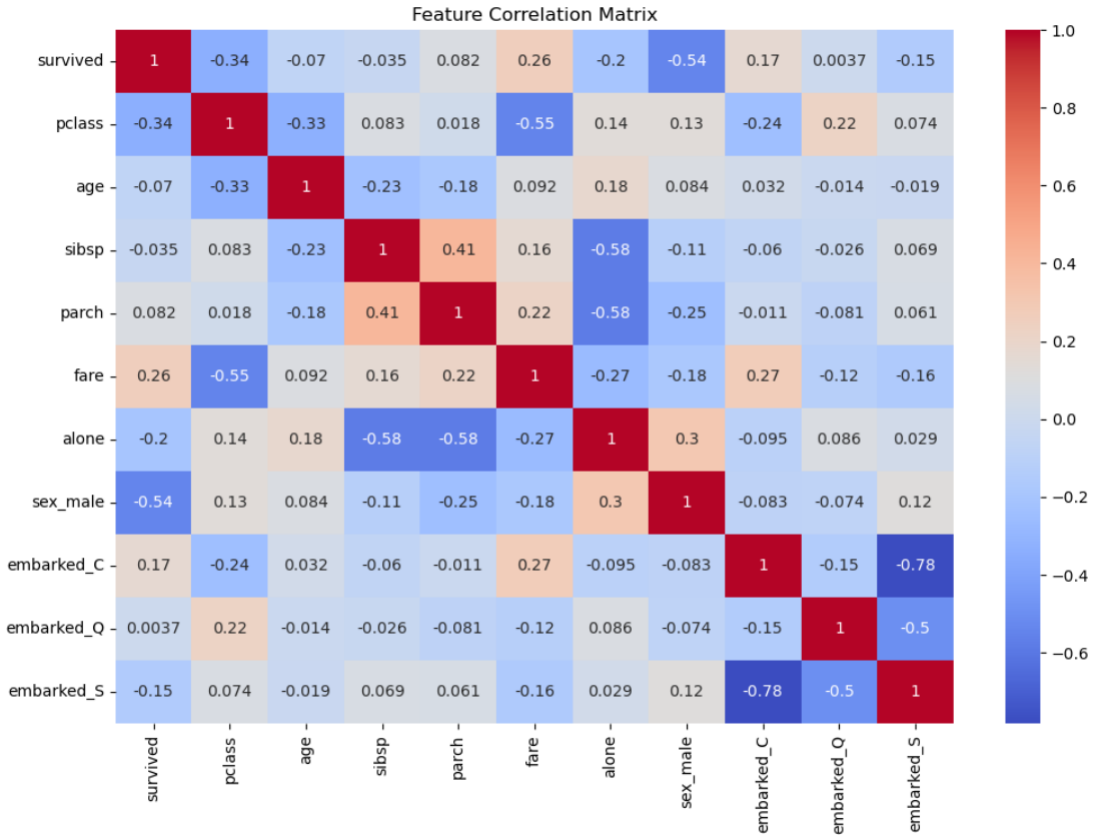
```
# 1. Define which columns were not encoded
non_encoded_columns = df2.drop(columns=binary_categorical_columns + multi_categorical_columns)

# 2. concat
df2_cleaned = pd.concat([non_encoded_columns, df2_binary_encoded, df2_multi_encoded], axis=1)
```

I convert categorical variables into numerical representations by using one-hot encoding technique. For example, column sex become sex_male with only two value: 0 represent female, 1 represent male.

k. Perform feature selection to identify relevant columns for analysis and machine learning.

```
# k. Perform feature selection to identify relevant columns for analysis and machine learning.
import seaborn as sns
import matplotlib.pyplot as plt


plt.figure(figsize=(12, 8))
sns.heatmap(df2_cleaned.corr(), annot=True, cmap='coolwarm')
plt.title("Feature Correlation Matrix")
plt.show()
```

To perform feature selection, I visualize the relationship of each columns by using heat map, the number in the map represent correlation coefficient which is between -1 and 1. The higher the coefficient, the larger the relationship between them.




```
from sklearn.feature_selection import SelectKBest, f_classif

X = df2_cleaned.drop("survived", axis=1)
y = df2_cleaned["survived"]

selector = SelectKBest(score_func=f_classif, k=5)
X_new = selector.fit_transform(X, y)


selected_features = X.columns[selector.get_support()]
print("Top features:", selected_features)
```

```
Top features: Index(['pclass', 'fare', 'alone', 'sex_male', 'embarked_C'], dtype='object')
```

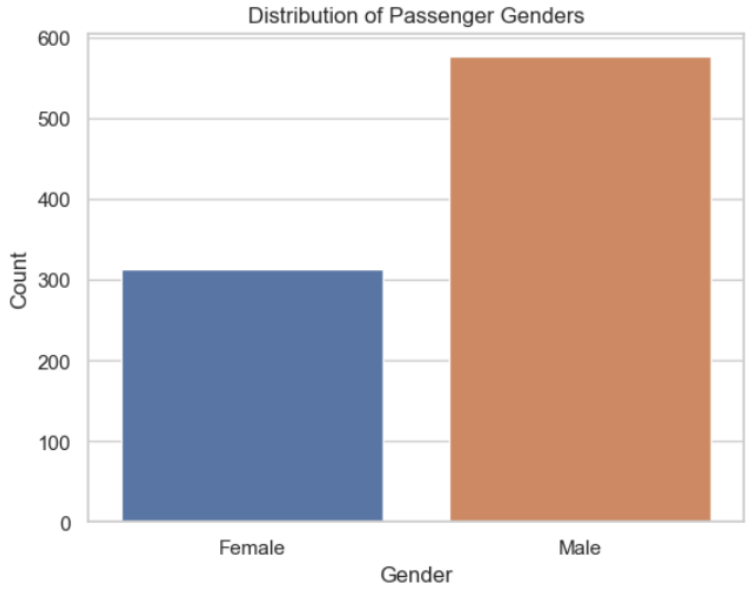
The top relevant feature is ‘pclass’, ‘fare’, ‘sex_male’, and ‘embarked_C’.

2. Data Visualization

Use Seaborn and Matplotlib to explore patterns in the **titanic** dataset from seaborn (use the pre-processed dataset from section C). Interpret insights from the visualizations.

a. Visualize the distribution of passengers’ genders (sex)

```
#a. Visualize the distribution of passengers' genders (sex)
sns.countplot(x='sex_male', data=df2_cleaned)
plt.xticks([0, 1], ['Female', 'Male'])
plt.title("Distribution of Passenger Genders")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.show()
```

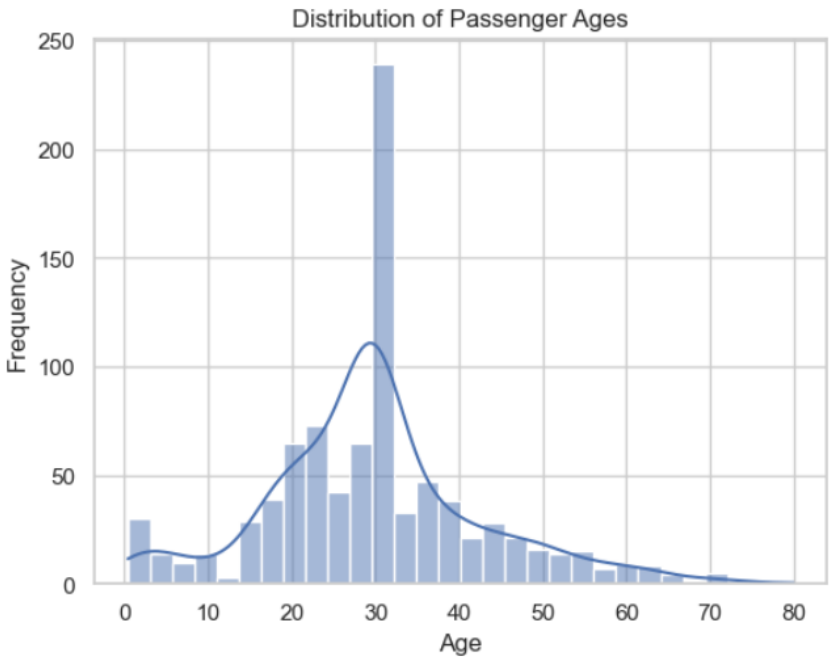


Since gender is a categorical variable, so I use count plot to visualize the distribution.

The count plot shows that male is more than female. There is around 320 female and around 580 male.

b. Show the distribution of passengers' ages (age).

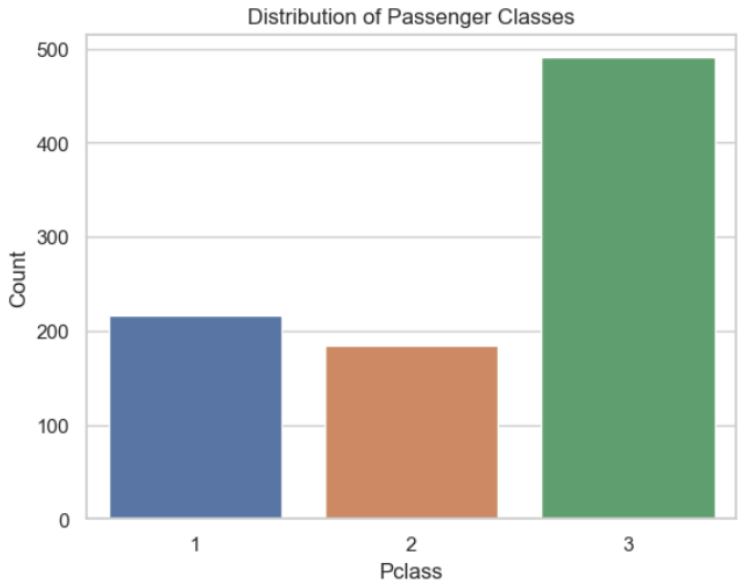
```
#b. Show the distribution of passengers' ages (age).
sns.histplot(data=df2_cleaned, x='age', bins=30, kde=True)
plt.title("Distribution of Passenger Ages")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```



Since age is a continuous numerical variable, so I use histogram to visualize the distribution. The histogram shows the distribution of passenger's age, while most passengers are in their early 30 years old.

c. Display the number of passengers in different classes (pclass).

```
#c. Display the number of passengers in different classes (pclass).
sns.countplot(x='pclass', data=df2_cleaned)
plt.title("Distribution of Passenger Classes")
plt.xlabel("Pclass")
plt.ylabel("Count")
plt.show()
```

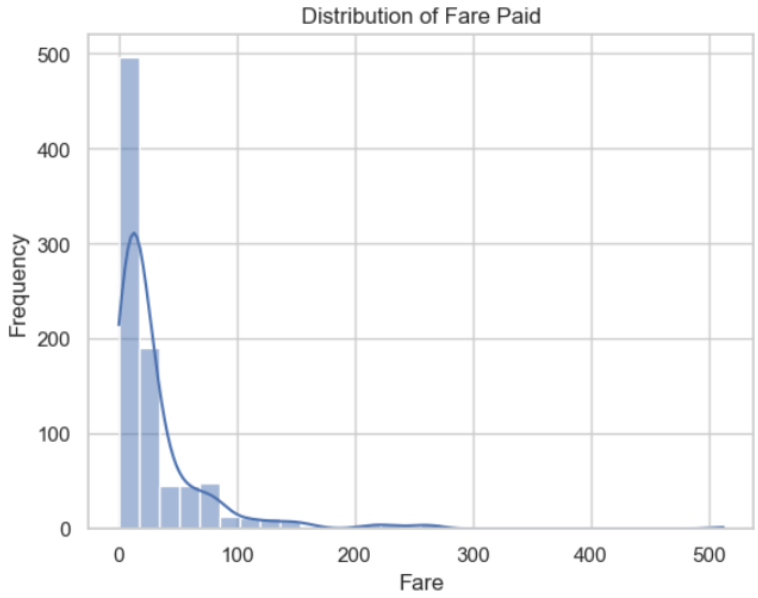


Since pclass is a categorical variable, so I use count plot to visualize the distribution.

Most of the people are class 3 (around 490), while the second is class 1 (around 220), the last is class 2 (around 180). Pclass is an ordinal categorical variable. The smaller the number, the higher the level. The ticket price is usually higher, and the passenger survival probability is often related to this level.

d. Show the distribution of fares paid (fare).

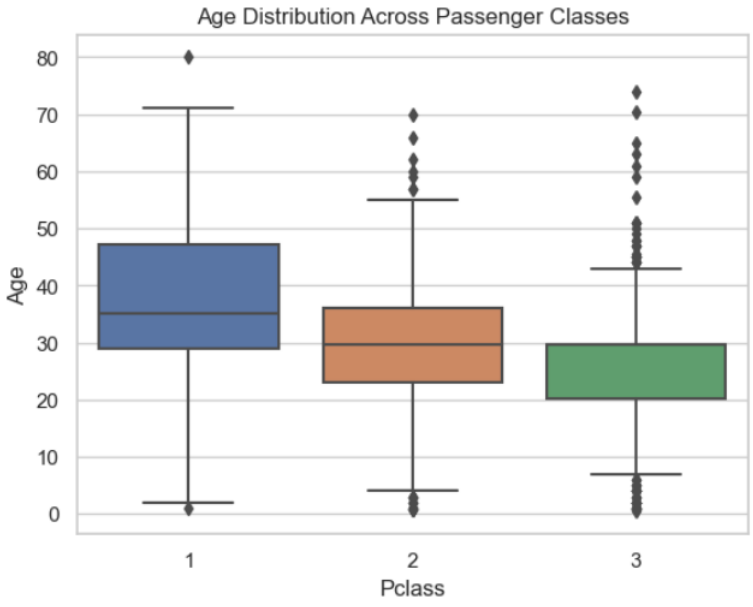
```
#d. Show the distribution of fares paid (fare)
sns.histplot(data=df2_cleaned, x='fare', bins=30, kde=True)
plt.title("Distribution of Fare Paid")
plt.xlabel("Fare")
plt.ylabel("Frequency")
plt.show()
```



Since fares paid is a continuous numerical variable, so I use histogram to visualize the distribution. Fares paid is not normally distributed, most people pay very little. The graph is a right skewed graph.

e. Compare age distributions across different passenger classes (pclass).

```
#e. Compare age distributions across different passenger classes (pclass).
sns.boxplot(x='pclass', y='age', data=df2_cleaned)
plt.title("Age Distribution Across Passenger Classes")
plt.xlabel("Pclass")
plt.ylabel("Age")
plt.show()
```

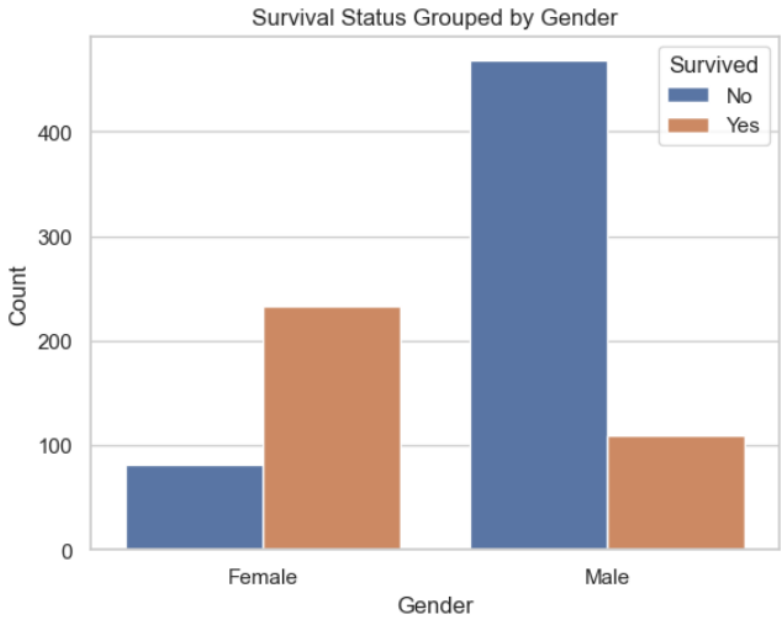


Since I want to compare age distributions (quantitative variable) across different passenger classes (categorical variable) , so I use boxplot to visualize the distribution.

The box plot shows that most people in class 1 are older, while people in class 3 are younger. The interquartile range of class 1 is the most widest, while the are the most outliers appear in class 3.

f. Show survival status grouped by gender (sex).

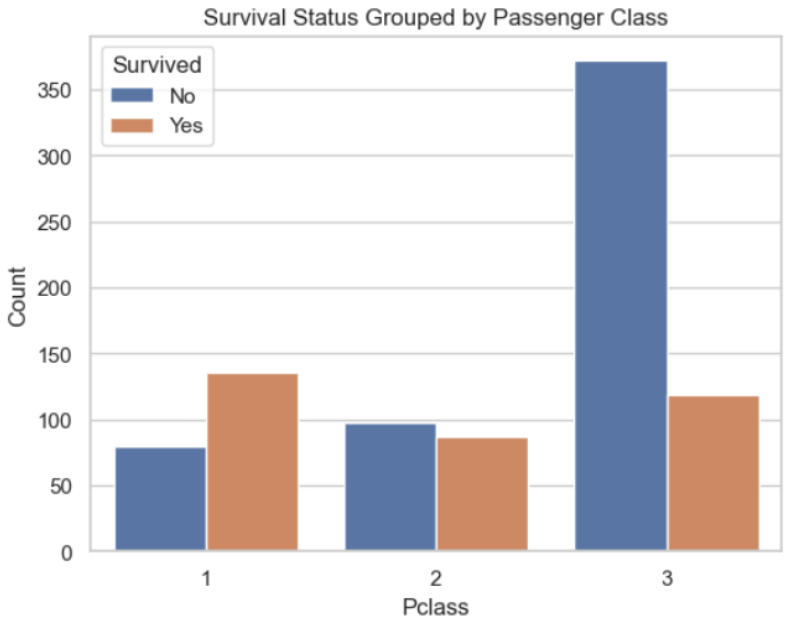
```
#f. Show survival status grouped by gender (sex).
sns.countplot(x='sex_male', hue='survived', data=df2_cleaned)
plt.xticks([0, 1], ['Female', 'Male'])
plt.title("Survival Status Grouped by Gender")
plt.xlabel("Gender")
plt.ylabel("Count")
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```



Since survival status and gender are both categorical variable, so I use count plot to visualize the distribution. The counter plot shows that male have lower survival rates than female.

g. Show survival status grouped by passenger class (pclass).

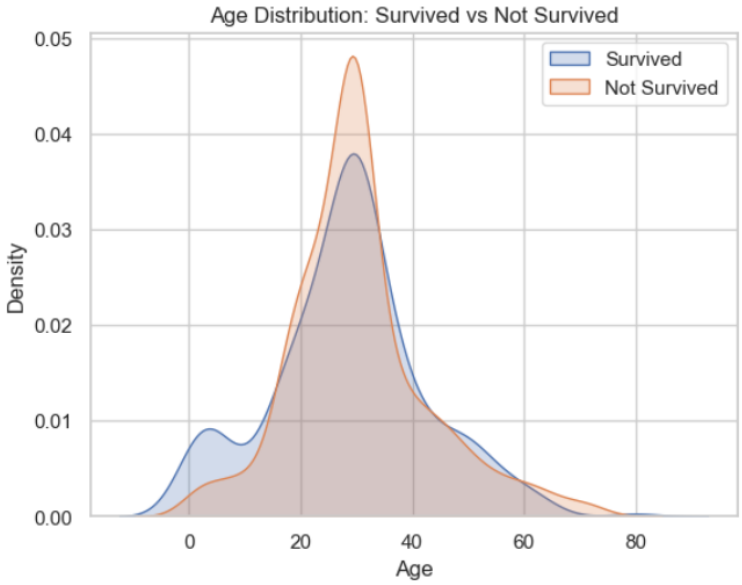
```
#g. Show survival status grouped by passenger class (pclass).
sns.countplot(x='pclass', hue='survived', data=df2_cleaned)
plt.title("Survival Status Grouped by Passenger Class")
plt.xlabel("Pclass")
plt.ylabel("Count")
plt.legend(title='Survived', labels=['No', 'Yes'])
plt.show()
```



Since survival status and passenger class are both categorical variable, so I use count plot to visualize the distribution. The counter plot shows that higher class (Pclass=1) has a higher survival rate. High-fare passengers are mostly from high-class seats and have a higher chance of survival.

h. Compare the age distribution of survivors and non-survivors.

```
#h. Compare the age distribution of survivors and non-survivors.
sns.kdeplot(data=df2_cleaned[df2_cleaned['survived'] == 1]['age'], label='Survived', shade=True)
sns.kdeplot(data=df2_cleaned[df2_cleaned['survived'] == 0]['age'], label='Not Survived', shade=True)
plt.title("Age Distribution: Survived vs Not Survived")
plt.xlabel("Age")
plt.ylabel("Density")
plt.legend()
plt.show()
```



Since age is numerical variable, I want to compare the age distribution of survivors and non-survivors (categorical variable), I decide to use sns.kdeplot() to visualize it.

Children (younger age) have a relatively higher survival rate, while people who are between 20 to 35 have a relatively lower survival rate.

Section D: Machine Learning & Model Evaluation

1. Supervised Learning - Passenger Survival Classification (use the pre-processed dataset from section C):

   Train a classification model to predict whether a passenger survived (survived column).

   a. Split the dataset into training (80%) and testing (20%) sets.

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from sklearn.linear_model import LogisticRegression
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report
```

```
#1.
#a. Split the dataset into training (80%) and testing (20%) sets.
# Define features and target
X = df2_cleaned.drop(columns=["survived"])
y = df2_cleaned["survived"]

#Normalization
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.2, random_state=42)
```

First, I import some necessary library to the environment, define feature variable X (all columns without ‘survived’) and target variable y (survived) . Then, I

standardized the feature variable, then split them into training (80%) and testing (20%) set.

b. Choose two machine learning algorithms to train a classification model.

c. Evaluate the model using the confusion matrix.

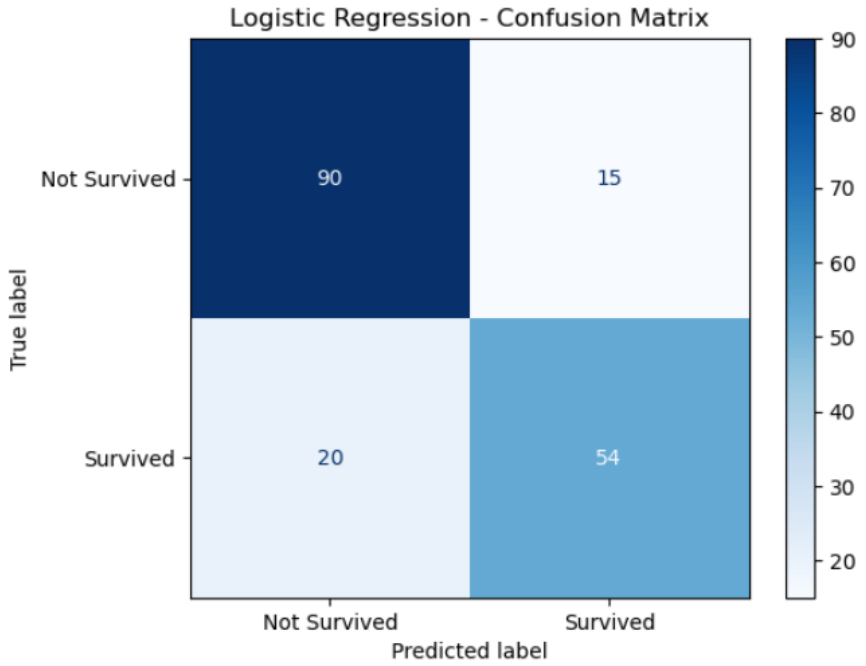
d. Compare the two models in terms of performance and discuss their limitations.

```
import matplotlib as mpl
mpl.rcParams.update(mpl.rcParamsDefault)
```

```
#b. Choose two machine learning algorithms to train a classification model.
#c. Evaluate the model using the confusion matrix.
# Build 2 models
models = {
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "Random Forest": RandomForestClassifier(random_state=42)
}

# Fit, predict, and evaluate both models
for name, model in models.items():
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    acc = accuracy_score(y_test, y_pred)
    cm = confusion_matrix(y_test, y_pred)

    print(f" {name}")
    print(f"Accuracy: {acc:.4f}")
    print("Confusion Matrix:")
    print(cm)


    tn, fp, fn, tp = cm.ravel()
    print(f"True Negatives: {tn}, False Positives: {fp}, False Negatives: {fn}, True Positives: {tp}")

    # Precision, Recall, F1-Score)
    print("Classification Report:")
    print(classification_report(y_test, y_pred, target_names=["Not Survived", "Survived"]))


    # Plot confusion matrix
    disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=["Not Survived", "Survived"])
    disp.plot(cmap="Blues")
    plt.title(f"{name} - Confusion Matrix")
    plt.show()
```

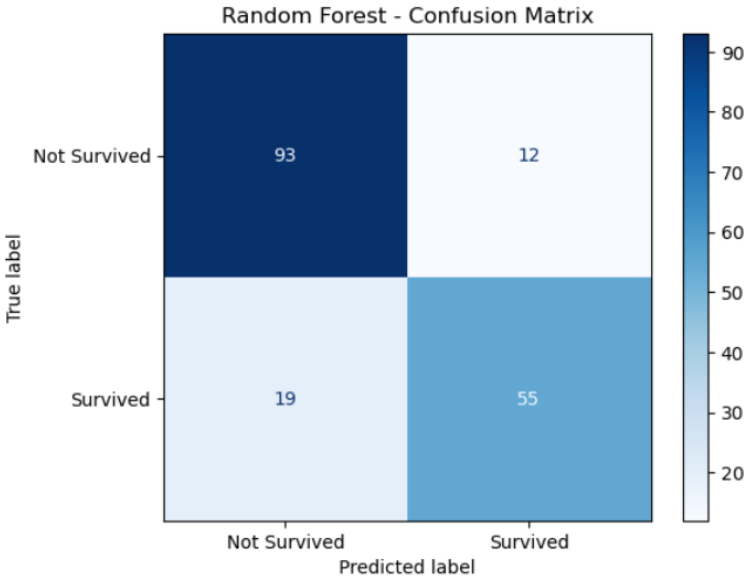
Logistic Regression model and Random Forest model has been chosen to train a classification model.

```
 Logistic Regression
Accuracy: 0.8045
Confusion Matrix:
[[90 15]
 [20 54]]
True Negatives: 90, False Positives: 15, False Negatives: 20, True Positives: 54
Classification Report:
              precision    recall  f1-score   support

Not Survived       0.82      0.86      0.84       105
    Survived       0.78      0.73      0.76        74

    accuracy                           0.80       179
   macro avg       0.80      0.79      0.80       179
weighted avg       0.80      0.80      0.80       179
```



For Logistic Regression model, accuracy is 80.45%, the confusion matrix shows that there are 90 true negatives, 15 false positive, 20 false negative, 54 true positive.

```
 Random Forest
Accuracy: 0.8268
Confusion Matrix:
[[93 12]
 [19 55]]
True Negatives: 93, False Positives: 12, False Negatives: 19, True Positives: 55
Classification Report:
              precision    recall  f1-score   support

Not Survived       0.83      0.89      0.86       105
    Survived       0.82      0.74      0.78        74

    accuracy                           0.83       179
   macro avg       0.83      0.81      0.82       179
weighted avg       0.83      0.83      0.83       179
```



For Random Forest model, accuracy is 82.68%, the confusion matrix shows that there are 93 true negatives, 12 false positive, 19 false negative, 55 true positive.

Logistic Regression performs well on linearly separable data and is interpretable, but might underfit complex relationships.

Random Forest generally has better performance on complex and nonlinear data, is robust to overfitting, but lacks interpretability and may be slower.

Performance Comparison：

Random Forest generally outperforms Logistic Regression when dealing with complex, non-linear relationships between features, as it is an ensemble method that combines multiple decision trees for higher accuracy and robustness. It tends to have better performance on classification tasks, especially when the dataset contains interactions or non-linear patterns that Logistic Regression may not capture.

Limitations:

Logistic Regression is easy to interpret and fast to train, but it assumes a linear relationship between features and the log-odds of the outcome, which can limit its accuracy. On the other hand, Random Forest is more powerful but less interpretable, and it can be computationally expensive on large datasets. It may also overfit if not properly tuned.

2. Unsupervised Learning - Passenger Clustering (use the pre-processed dataset from section C).

   Apply K-Means clustering to group passengers based on age, fare, and passenger class (pclass).

   a. Select age, fare, and class as clustering features.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
```

```
#a. Select age, fare, and class as clustering features.
clustering_features = df2_cleaned[["age", "fare", "pclass"]]
```

age: reflects the age distribution of passengers, which may affect clustering (children vs adults)

fare: shows the passenger's ability to pay, which is related to economic background

pclass: cabin class, reflects socioeconomic status, and is closely related to fare

b. Scale the data for better clustering performance.

```
# b. Scale the data for better clustering performance.
scaler = StandardScaler()
scaled_features = scaler.fit_transform(clustering_features)
```

K-Means performs clustering based on distance, and if it is not standardized, features with large unit differences, such as fare will dominate the clustering results.

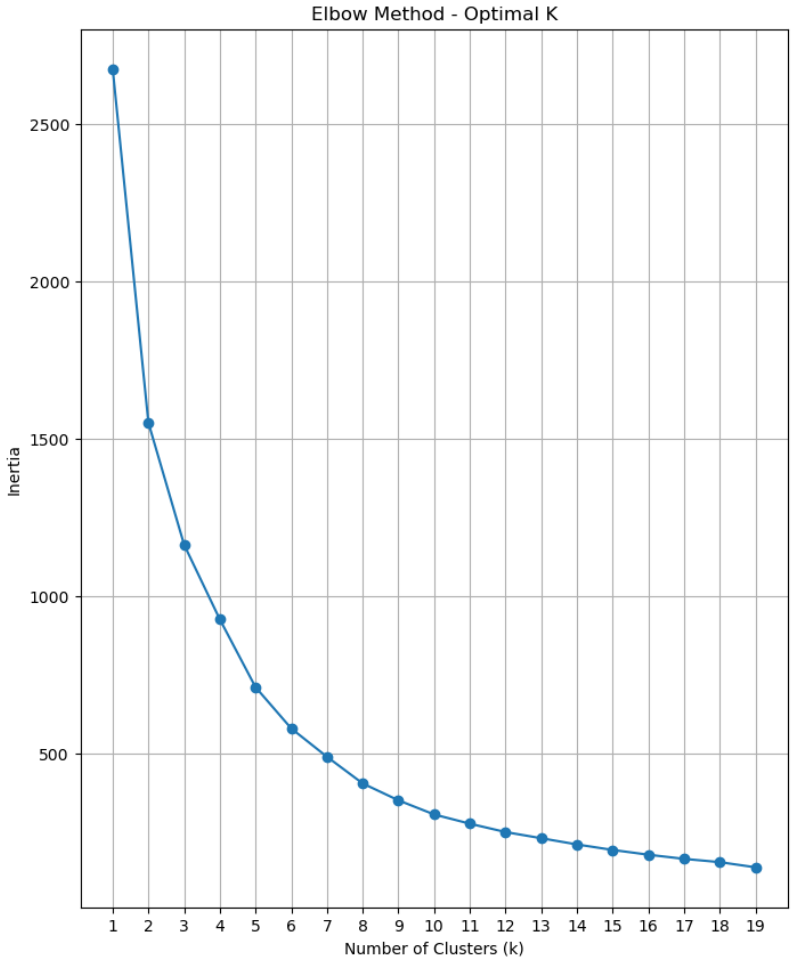
c. Apply K-Means clustering

- Experiment with different values of k (number of clusters).
- Use the Elbow Method to determine the optimal k.

```
#c. Apply K-Means clustering
inertia = []
k_range = range(1, 20)

for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(scaled_features)
    inertia.append(kmeans.inertia_)
```

For k=1 to 20, we apply the K-Means clustering and record its inertia (Within-group sum of squares error).

```
plt.figure(figsize=(8, 10))
plt.plot(k_range, inertia, marker='o')
plt.title("Elbow Method - Optimal K")
plt.xlabel("Number of Clusters (k)")
plt.ylabel("Inertia")
plt.xticks(k_range)
plt.grid(True)
plt.show()
```



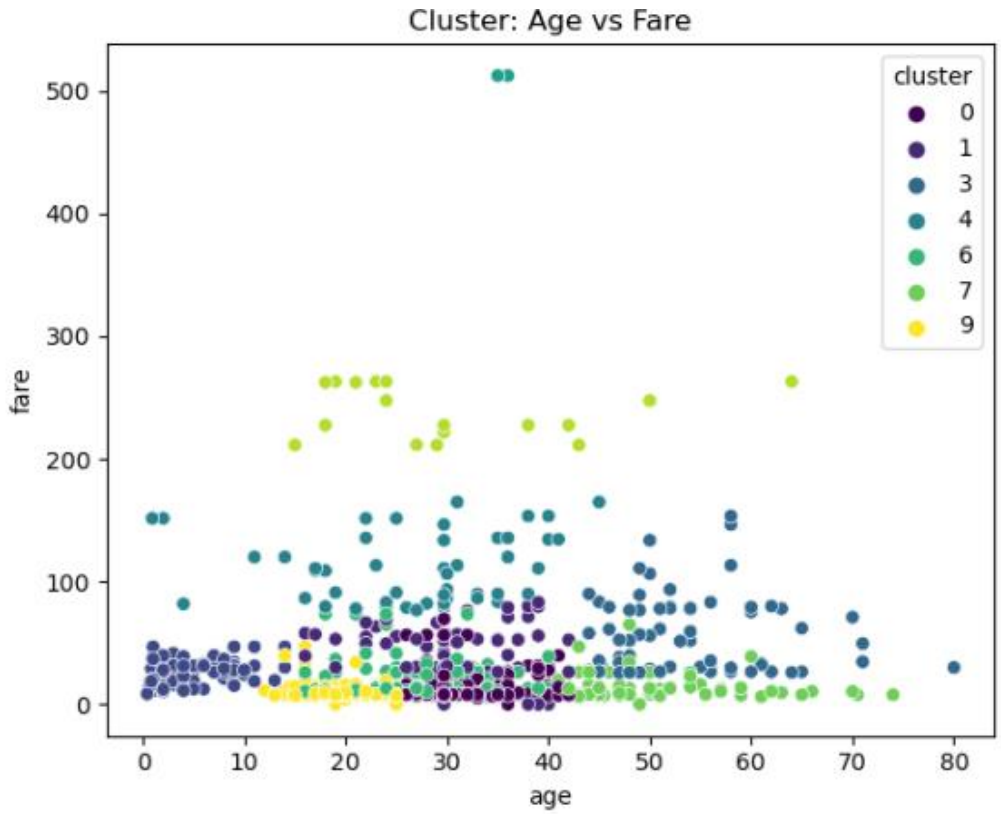
From the graph, I can observed that when k = 10, a inflection point appears (the error decreases slowly), it means that this k is a reasonable choice, so we use k = 10 to train it again.

```
optimal_k = 10
kmeans_final = KMeans(n_clusters=optimal_k, random_state=42, n_init=10)
cluster_labels = kmeans_final.fit_predict(scaled_features)
```

```
df2_cleaned["cluster"] = cluster_labels
```

d. Visualize the clusters using a scatter plot.

```
plt.figure(figsize=(18, 5))

# Age vs Fare
plt.subplot(1, 3, 1)
sns.scatterplot(data=df2_cleaned, x='age', y='fare', hue='cluster', palette='viridis')
plt.title('Cluster: Age vs Fare')

# Age vs Pclass
plt.subplot(1, 3, 2)
sns.scatterplot(data=df2_cleaned, x='age', y='pclass', hue='cluster', palette='viridis')
plt.title('Cluster: Age vs Pclass')

# Fare vs Pclass
plt.subplot(1, 3, 3)
sns.scatterplot(data=df2_cleaned, x='fare', y='pclass', hue='cluster', palette='viridis')
plt.title('Cluster: Fare vs Pclass')

plt.tight_layout()
plt.show()
```

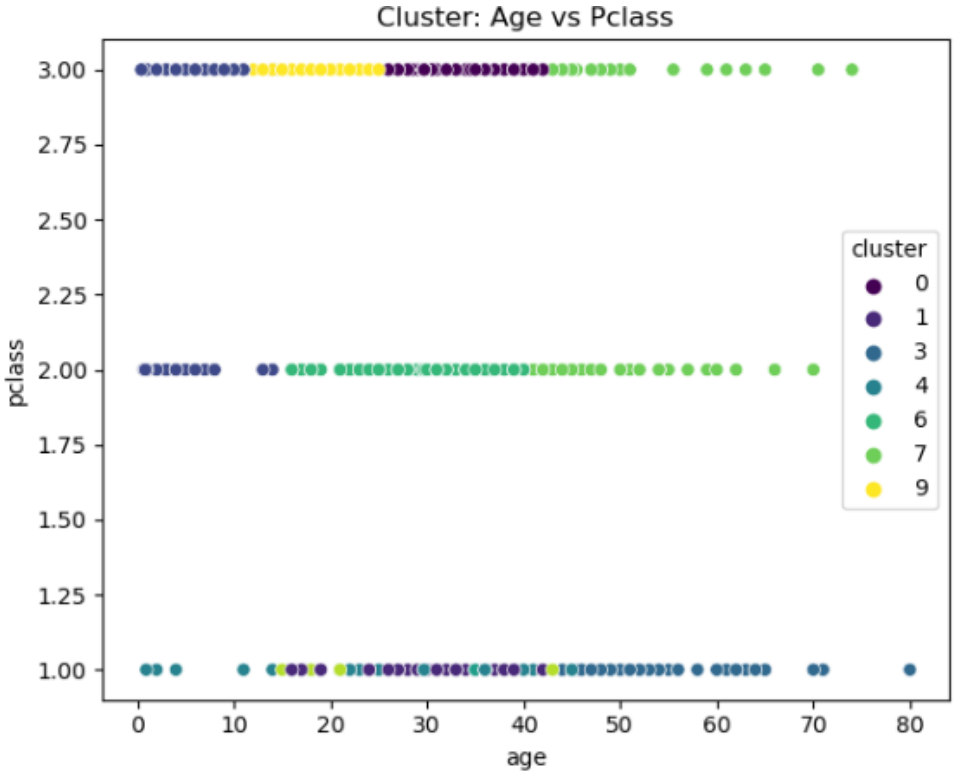


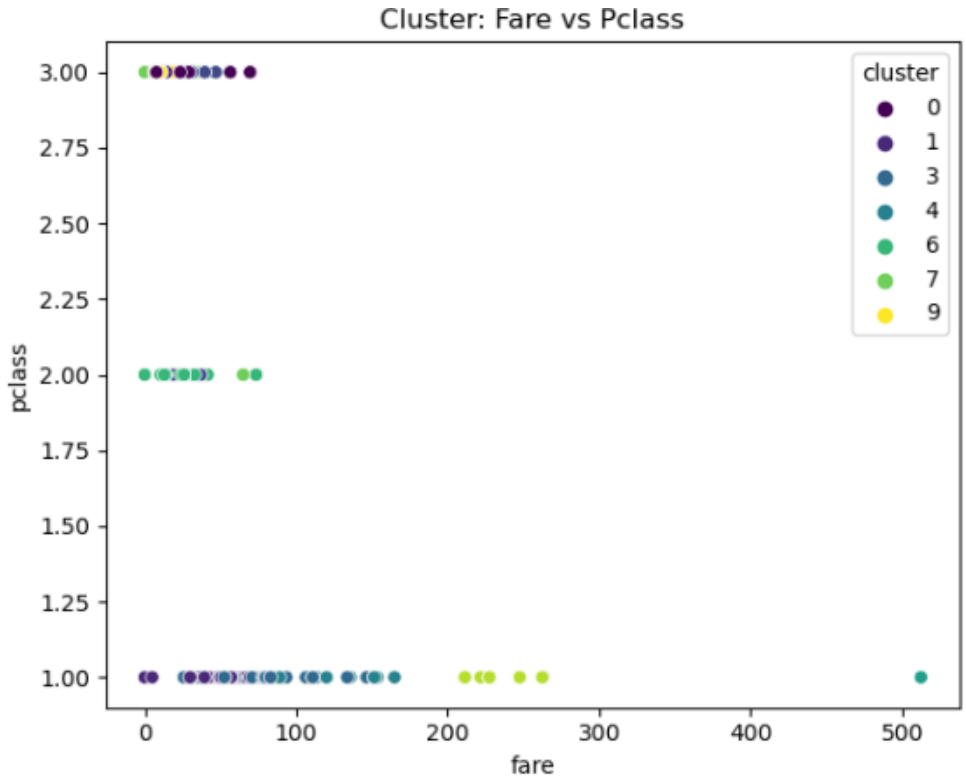
Using a scatter plot to display the clustering of passengers on the age-fare plane helps determine whether the clustering is effective and the characteristic distribution of different clusters.

We often choose the two most "representative" variables, such as fare vs age, to see if the cluster distribution is obvious.

In cluster visualization, the two "most representative" variables usually refer to variables with a large range of numerical changes in the sample, continuous data distribution, and good ability to distinguish different clusters.

In this data set, age is a continuous variable with a large range of values, fare is another continuous variable with a rich range of values. They jointly determine the relationship between each individual's age and fare, and naturally become the core basis for clustering.





P class is not a continuous numerical variable, cannot reflect continuous clustering structure, so pclass vs age and pclass vs fare is not the most "representative" variables, is not appropriate for cluster visualization.

APPENDIX 1

MARKING RUBRICS

Component Title	Assignment				Total Mark	100	
Criteria	Score and Descriptors					Weight (%)	Marks
	Excellent (9-8)	Good (7-5)	Average (4-3)	Need Improvement (2)	Poor (0-1)		
Section A	The source code fully addresses the question without compilation error. It has sensible variables.	The source code addresses the question without compilation error.	The source code partially addresses the question, but the program still works.	The source code partially addresses the question and has a compilation error.	The source code doesn't work and doesn't address the question. Or No Attempt	9	
	Excellent (14-11)	Good (10-8)	Average (7-5)	Need Improvement (4-3)	Poor (2-0)		
Section B	The source code fully addresses the question without compilation error. It has sensible variables.	The source code addresses the question without compilation error.	The source code partially addresses the question, but the program still works.	The source code partially addresses the question and has a compilation error.	The source code doesn't work and doesn't address the question. Or No Attempt	14	
	Excellent (23-19)	Good (18-14)	Average (13-8)	Need Improvement (7-4)	Poor (0-3)		
Section C	The source code fully addresses the question without compilation error. It has sensible variables.	The source code addresses the question without compilation error.	The source code partially addresses the question, but the program still works.	The source code partially addresses the question and has a compilation error.	The source code doesn't work and doesn't address the question. Or No Attempt	23	
	Excellent (24-19)	Good (18-14)	Average (13-8)	Need Improvement (7-4)	Poor (0-3)		
Section D	The source code fully addresses the question without compilation error. It has sensible variables.	The source code addresses the question without compilation error.	The source code partially addresses the question, but the program still works.	The source code partially addresses the question and has a compilation error.	The source code doesn't work and doesn't address the question. Or No Attempt	24	
					Subtotal	70	

	Excellent (30-24)	Good (23-18)	Average (17-14)	Need Improvement (13-8)	Poor (7-0)		
Report	The report meets all criteria exceptionally well.	The report meets most criteria well but has minor issues.	The report meets some criteria but has noticeable issues.	The report meets few criteria and has significant issues.	The report does not meet the criteria or is incomplete. Or No Attempt	30	
					Subtotal	30	
					TOTAL	100	

Note to students: Please include the marking rubric when submitting your coursework.